

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Жизненный цикл разработки программного обеспечения

Отчёт по лабораторной работе №1
«Интегрированная среда разработки ПО»

Выполнил: Родкевич Н.А.
Группа: 434701
Проверил: Внук О.М.

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы: – изучить методы работы с одной из выбранных студентом сред разработки: IntelliJ IDEA (рекомендуемая среда), Eclipse, Microsoft Visual Studio Code, учитывая, что использовать можно только объектно-ориентированный язык программирования (C++, C#, Java, другой язык по согласованию с преподавателем); – научиться разрабатывать в выбранной среде приложение с графическим интерфейсом; – изучить MVC с активной и пассивной моделью.

Разработать программу Идеальное похудение. Задача программы – выдать пульс для кардиотренировки (велотренажер, степ тренажер, гребной тренажер, эллипсоид), при которой наиболее эффективно сжигаются жиры и вымываются токсины из мышц (в поиске ключевые слова Формула Карвонена \ рассчитать норму пульса во время тренировок). Человек вводит свой возраст, пульс в состоянии покоя. Программа выдает ему, какой пульс наиболее ему подходит для сжигания жиров. На главной форме разместить кнопку «Ввод данных», данные ввести на отдельной форме. Последние введенные данные нужно запоминать в главной форме и восстанавливать при повторном вводе. Если введены некорректные данные – показать сообщение об ошибке. Отобразить на главной форме пульс, с которого нужно начинать занятия кардиотренировками, и пульс, на который нужно выйти через несколько занятий. Добавить каркас MVC с активной моделью. Неуспевающим студентам применить MVC с пассивной моделью. Для совсем неуспевающих студентов – из паттерна MVC реализовать класс контроллер, совмещенный с пассивной моделью.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Интегрированная среда разработки, IDE (англ. Integrated development environment) – комплекс программных средств, используемых программистами для разработки программного обеспечения. Среда разработки включает в себя:

- текстовый редактор;
- компилятор и/или интерпретатор;
- средства автоматизации сборки;
- отладчик.

Иногда IDE содержит также средства для интеграции с системами управления версиями и разнообразные инструменты для упрощения конструирования графического интерфейса пользователя.

Многие современные среды разработки также включают браузер классов, инспектор объектов и диаграмму иерархии классов – для использования при объектно-ориентированной разработке ПО.

IDE обычно предназначены для нескольких языков программирования – такие как IntelliJ IDEA, NetBeans, Eclipse, Qt Creator,

Geany, Embarcadero RAD Studio, Code::Blocks, Xcode или Microsoft Visual Studio, но есть и IDE для одного определённого языка программирования – как, например, Visual Basic, Delphi, Dev-C++.

Основные компоненты IDE:

- редактор кода;
- средства сборки проекта;
- средства запуска проекта;
- средства отладки;
- средства тестирования;
- поддержка совместной работы;
- расширения.

Эффективная работа в IDE строится на быстрых клавишах, умном автодополнении, безопасном рефакторинге и удобной навигации. Эти приёмы экономят время и снижают количество ошибок при написании кода.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

В соответствии с вариантом задания была разработана программа «Идеальное похудение», которая на основании введённых данных рассчитывает тренировочный пульс и выводит на экран начальное и целевое значения. Требования варианта также предусматривают отдельную форму ввода, сохранение последних значений, восстановление данных и обработку некорректного ввода.

4 ЛИСТИНГ ИСХОДНОГО КОДА С КОММЕНТАРИЯМИ

```
package controller;

import model.WorkoutData;
import model.WorkoutModel;
import view.InputDialog;
import view.MainFrame;

import java.beans.PropertyChangeEvent;

public class WorkoutController {
    private final WorkoutModel model;
    private final MainFrame view;

    public WorkoutController(WorkoutModel model, MainFrame view) {
        this.model = model;
        this.view = view;
        view.setOpenInputAction(e -> openInputDialog());
        model.addPropertyChangeListener(this::onModelChanged);
        WorkoutData saved = model.getData();
        view.showLastInput(saved);
    }

    private void openInputDialog() {
        InputDialog dialog = new InputDialog(view, model.getData());
        dialog.setVisible(true);
        if (dialog.isConfirmed()) {
            WorkoutData data = dialog.getData();
            try {
                validate(data);
                model.setData(data);
                model.calculate();
            } catch (IllegalArgumentException ex) {
                view.showError(ex.getMessage());
            }
        }
    }
}
```

```

    }
}

private void validate(WorkoutData data) {
    if(data == null) {
        throw new IllegalArgumentException("Данные не введены.");
    }
    if(data.getAge() < 10 || data.getAge() > 100) {
        throw new IllegalArgumentException("Возраст должен находиться в диапазоне от 10 до 100 лет.");
    }
    if(data.getRestingHeartRate() < 30 || data.getRestingHeartRate() > 150) {
        throw new IllegalArgumentException("Пульс в состоянии покоя должен находиться в диапазоне от 30 до 150 уд/мин.");
    }
    int maxHeartRate = 220 - data.getAge();
    if(data.getRestingHeartRate() >= maxHeartRate) {
        throw new IllegalArgumentException("Пульс покоя должен быть меньше расчётного максимального пульса.");
    }
}

private void onModelChanged(PropertyChangeEvent event) {
    if("startPulse".equals(event.getPropertyName()) || "targetPulse".equals(event.getPropertyName())) {
        view.showResults(model.getData(), model.getStartPulse(), model.getTargetPulse());
    }
    if("data".equals(event.getPropertyName())) {
        view.showLastInput(model.getData());
    }
}
}

package model;

public final class WorkoutData {
    private final int age;
    private final int restingHeartRate;

    public WorkoutData(int age, int restingHeartRate) {
        this.age = age;
        this.restingHeartRate = restingHeartRate;
    }

    public int getAge() {
        return age;
    }

    public int getRestingHeartRate() {
        return restingHeartRate;
    }
}

package model;

import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.util.prefs.Preferences;

public class WorkoutModel {
    private static final double START_INTENSITY = 0.60;
    private static final double TARGET_INTENSITY = 0.70;

    private final PropertyChangeSupport support = new PropertyChangeSupport(this);
    private final Preferences preferences = Preferences.userNodeForPackage(WorkoutModel.class);

    private WorkoutData data;
    private int startPulse;
    private int targetPulse;

    public WorkoutModel() {
        int savedAge = preferences.getInt("age", 30);
        int savedRest = preferences.getInt("restingHeartRate", 70);
        data = new WorkoutData(savedAge, savedRest);
    }

    public void setData(WorkoutData newData) {
        WorkoutData oldData = this.data;
        this.data = newData;
        preferences.putInt("age", newData.getAge());
        preferences.putInt("restingHeartRate", newData.getRestingHeartRate());
    }
}

```

```

        support.firePropertyChange("data", oldData, newData);
    }

    public void calculate() {
        int oldStart = startPulse;
        int oldTarget = targetPulse;
        int maxHeartRate = 220 - data.getAge();
        int heartRateReserve = maxHeartRate - data.getRestingHeartRate();
        startPulse = roundPulse(data.getRestingHeartRate() + heartRateReserve * START_INTENSITY);
        targetPulse = roundPulse(data.getRestingHeartRate() + heartRateReserve * TARGET_INTENSITY);
        support.firePropertyChange("startPulse", oldStart, startPulse);
        support.firePropertyChange("targetPulse", oldTarget, targetPulse);
    }

    private int roundPulse(double value) {
        return (int) Math.round(value);
    }

    public WorkoutData getData() {
        return data;
    }

    public int getStartPulse() {
        return startPulse;
    }

    public int getTargetPulse() {
        return targetPulse;
    }

    public void addPropertyChangeListener(PropertyChangeListener listener) {
        support.addPropertyChangeListener(listener);
    }

    public void removePropertyChangeListener(PropertyChangeListener listener) {
        support.removePropertyChangeListener(listener);
    }
}

package view;

import model.WorkoutData;

import javax.swing.*;
import java.awt.*;

public class InputDialog extends JDialog {
    private final JTextField ageField = new JTextField(12);
    private final JTextField restingPulseField = new JTextField(12);
    private boolean confirmed;
    private WorkoutData data;
    public InputDialog(Frame owner, WorkoutData previousData) {
        super(owner, "Ввод данных", true);
        setSize(430, 240);
        setLocationRelativeTo(owner);
        if (previousData != null) {
            ageField.setText(String.valueOf(previousData.getAge()));
            restingPulseField.setText(String.valueOf(previousData.getRestingHeartRate()));
        }
        buildUi();
    }

    private void buildUi() {
        JPanel form = new JPanel(new GridLayout(2, 2, 10, 10));
        form.setBorder(BorderFactory.createEmptyBorder(20, 20, 10, 20));
        form.add(new JLabel("Возраст, лет:"));
        form.add(ageField);
        form.add(new JLabel("Пульс в покое, уд/мин:"));
        form.add(restingPulseField);
        JButton calculateButton = new JButton("Рассчитать");
        JButton cancelButton = new JButton("Отмена");
        calculateButton.addActionListener(e -> confirmInput());
        cancelButton.addActionListener(e -> dispose());
        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 10));
        buttons.add(calculateButton);
        buttons.add(cancelButton);
        setLayout(new BorderLayout());
        add(form, BorderLayout.CENTER);
    }
}

```

```

        add(buttons, BorderLayout.SOUTH);
        getRootPane().setDefaultButton(calculateButton);
    }

    private void confirmInput() {
        try {
            int age = Integer.parseInt(ageField.getText().trim());
            int restingPulse = Integer.parseInt(restingPulseField.getText().trim());
            data = new WorkoutData(age, restingPulse);
            confirmed = true;
            dispose();
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Введите целые числовые значения возраста и пульса.", "Ошибка ввода",
JOptionPane.ERROR_MESSAGE);
        }
    }

    public boolean isConfirmed() {
        return confirmed;
    }

    public WorkoutData getData() {
        return data;
    }
}

package view;

import model.WorkoutData;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionListener;

public class MainFrame extends JFrame {
    private final JLabel ageValue = new JLabel("-");
    private final JLabel restingPulseValue = new JLabel("-");
    private final JLabel startPulseValue = new JLabel("-");
    private final JLabel targetPulseValue = new JLabel("-");
    private final JButton inputButton = new JButton("Ввод данных");

    public MainFrame() {
        setTitle("Идеальное похудение");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(620, 420);
        setLocationRelativeTo(null);
        setMinimumSize(new Dimension(560, 360));
        buildUi();
    }

    private void buildUi() {
        JPanel root = new JPanel(new BorderLayout(15, 15));
        root.setBorder(BorderFactory.createEmptyBorder(20, 25, 20, 25));
        JLabel title = new JLabel("Идеальное похудение", SwingConstants.CENTER);
        title.setFont(new Font("SansSerif", Font.BOLD, 24));
        JLabel description = new JLabel("<html><center>Расчёт тренировочного пульса по формуле Карвонена<br> + \"для интенсивности 60–70%</center></html>", SwingConstants.CENTER);
        JPanel header = new JPanel(new BorderLayout(5, 10));
        header.add(title, BorderLayout.NORTH);
        header.add(description, BorderLayout.SOUTH);
        JPanel values = new JPanel(new GridLayout(4, 2, 12, 12));
        values.setBorder(BorderFactory.createTitledBorder("Результат"));
        addRow(values, "Возраст, лет:", ageValue);
        addRow(values, "Пульс в покое, уд/мин:", restingPulseValue);
        addRow(values, "Начальный пульс (60%), уд/мин:", startPulseValue);
        addRow(values, "Целевой пульс (70%), уд/мин:", targetPulseValue);
        JPanel bottom = new JPanel(new FlowLayout(FlowLayout.CENTER));
        inputButton.setPreferredSize(new Dimension(180, 38));
        bottom.add(inputButton);
        root.add(header, BorderLayout.NORTH);
        root.add(values, BorderLayout.CENTER);
        root.add(bottom, BorderLayout.SOUTH);
        setContentPane(root);
    }

    private void addRow(JPanel panel, String label, JLabel value) {
        panel.add(new JLabel(label));
        value.setHorizontalAlignment(SwingConstants.RIGHT);
    }
}

```

```

        value.setFont(new Font("SansSerif", Font.BOLD, 15));
        panel.add(value);
    }

    public void setOpenInputAction(ActionListener listener) {
        inputButton.addActionListener(listener);
    }

    public void showLastInput(WorkoutData data) {
        if (data != null) {
            ageValue.setText(String.valueOf(data.getAge()));
            restingPulseValue.setText(String.valueOf(data.getRestingHeartRate()));
        }
    }

    public void showResults(WorkoutData data, int startPulse, int targetPulse) {
        showLastInput(data);
        startPulseValue.setText(String.valueOf(startPulse));
        targetPulseValue.setText(String.valueOf(targetPulse));
    }

    public void showError(String message) {
        JOptionPane.showMessageDialog(this, message, "Ошибка ввода", JOptionPane.ERROR_MESSAGE);
    }
}

import controller.WorkoutController;
import model.WorkoutModel;
import view.MainFrame;

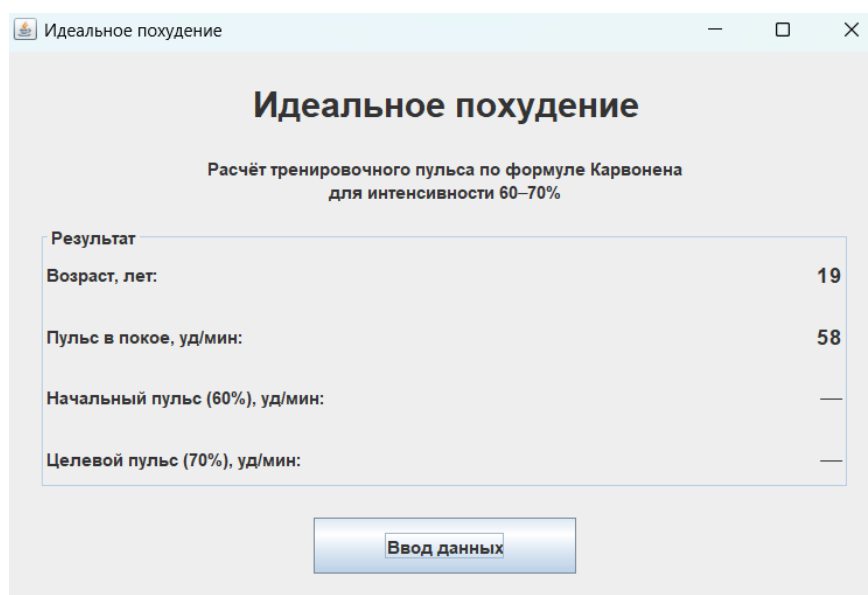
import javax.swing.*;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            WorkoutModel model = new WorkoutModel();
            MainFrame view = new MainFrame();
            new WorkoutController(model, view);
            view.setVisible(true);
        });
    }
}

```

5 СКРИНШОТЫ РАБОТАЮЩЕГО ПРИЛОЖЕНИЯ

На представленных ниже скриншотах показана работа разработанного приложения. На рисунке 5.1 показано главное окно приложения.



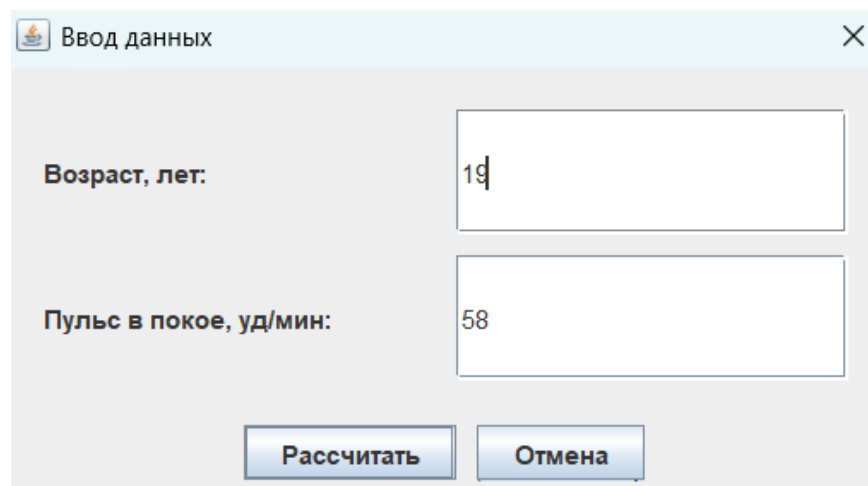
The screenshot shows the main window of the application. The title bar reads 'Идеальное похудение'. The main content area has a title 'Идеальное похудение' and a subtitle 'Расчёт тренировочного пульса по формуле Карвонена для интенсивности 60–70%'. Below this, there is a table with the following data:

Результат	
Возраст, лет:	19
Пульс в покое, уд/мин:	58
Начальный пульс (60%), уд/мин:	—
Целевой пульс (70%), уд/мин:	—

At the bottom center, there is a button labeled 'Ввод данных'.

Рисунок 5.1 – Главное окно приложения

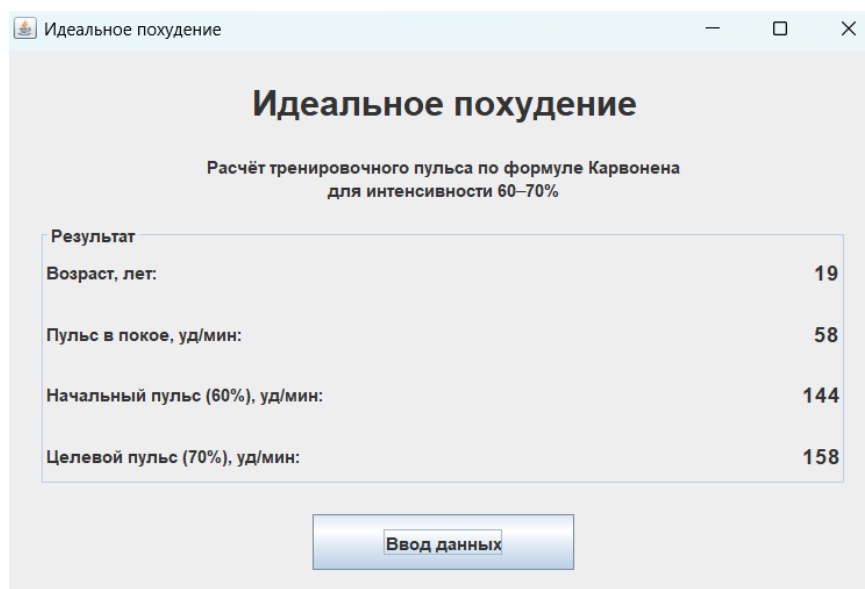
На рисунке 5.2 показано диалоговое окно ввода данных.



The screenshot shows a dialog box titled 'Ввод данных'. It contains two input fields: 'Возраст, лет:' with the value '19' and 'Пульс в покое, уд/мин:' with the value '58'. At the bottom, there are two buttons: 'Рассчитать' and 'Отмена'.

Рисунок 5.2 – Диалоговое окно ввода данных

На рисунке 5.3 показаны результаты расчётов.



The screenshot shows a window titled "Идеальное похудение" with a subtitle "Расчёт тренировочного пульса по формуле Карвонена для интенсивности 60–70%". It displays a table of results and a button labeled "Ввод данных".

Результат	
Возраст, лет:	19
Пульс в покое, уд/мин:	58
Начальный пульс (60%), уд/мин:	144
Целевой пульс (70%), уд/мин:	158

Ввод данных

Рисунок 5.3 – Результаты расчёта

На рисунках 5.4 и 5.5 показаны уведомления о некорректных исходных данных.

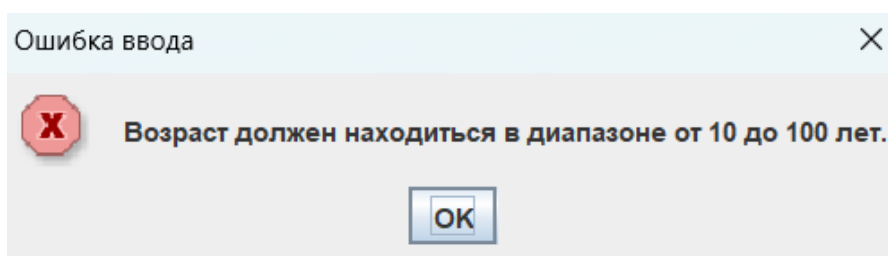


Рисунок 5.4 – Некорректный ввод возраста

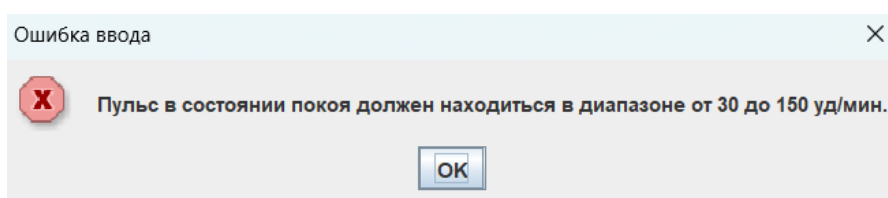


Рисунок 5.5 – Некорректный ввод пульса

На приведённых выше рисунках показаны все возможные варианты результатов работы приложения.

6 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Основные компоненты IDE:

- редактор кода;
- средства сборки проекта;
- средства запуска проекта;
- средства отладки;
- средства тестирования;
- поддержка совместной работы;
- расширения.

Основные приемы работы с редактором кода включают навигацию, автодополнение, поиск, мультикурсорное редактирование и автоматическое форматирование.

Сборка и запуск программного проекта выполняются с помощью специального инструмента управления сборкой (например, Maven, Gradle, mpm, Make) в зависимости от используемого языка программирования и технологий.

MVC – это популярный архитектурный шаблон, который делит программу на три независимых компонента, чтобы отделить данные от интерфейса.

Из чего состоит MVC:

- модель (Model): хранит данные и отвечает за бизнес-логику (правила работы программы, запросы к базе данных);
- представление (View): отвечает за визуальное отображение данных для пользователя (интерфейс, HTML-страницы, кнопки, текст);
- контроллер (Controller): выступает связующим звеном. Он принимает действия пользователя, передает команды Модели и выбирает, какое Представление показать в ответ.

Главное отличие между активной и пассивной моделью в архитектуре MVC заключается в том, кто именно сообщает Представлению (View) об изменении данных.

В активной модели данные «общаются» с интерфейсом напрямую через специальный механизм подписки, а в пассивной – всё взаимодействие происходит строго через Контроллер.